Resilient TaskFlow: A Crash-Safe Job Processing System Using a Transactional Outbox, Lease Fencing, and Redis Streams

Pankaj Baid
baidpankaj567@gmail.com

Abstract—Asynchronous job systems commonly lose jobs when a process crashes between saving a job and queueing it, or run a job twice after a worker dies. Resilient TaskFlow is a TypeScript job queue that keeps PostgreSQL as the only source of truth and uses Redis Streams only to notify workers. It combines a transactional outbox, time-limited leases with fencing, acknowledgement after commit, and a priority window ordered in PostgreSQL. Delivery is at-least-once, so handlers must make external effects idempotent. We describe the design, map each failure scenario to an automated test, and report measurements from a single laptop. One API process admitted 500 jobs/s with a 346 ms p99 and no errors. Draining a 20,000-job backlog reached about 200 to 250 jobs/s regardless of worker count because PostgreSQL was the bottleneck. After a SIGKILL of one of two workers, no job was lost and 8 of 10,000 jobs were re-run after the 30 s lease; after SIGTERM, none consumed an extra attempt. Benchmarking also exposed three defects that we fixed. These are single-machine results, not production capacity.

Index Terms—job queue, transactional outbox, lease fencing, at-least-once delivery, PostgreSQL, Redis Streams, fault tolerance

I. Introduction

Background work such as sending email or processing uploads is usually handed to a queue. Two failures recur in practice. First, a service writes a job to its database and then publishes it to a broker; a crash between the two leaves a job that is saved but never runs. Second, a worker takes a job, crashes, and the job is either lost (the broker already removed it) or run again with no record that a first attempt happened.

Resilient TaskFlow addresses both with a deliberately small set of mechanisms and tests each against real PostgreSQL and Redis processes. The contributions are:

- A design in which PostgreSQL alone decides job state, and a transactional outbox makes "job saved" and "job will be delivered" a single atomic fact (Section III).
- Fenced leases and acknowledgement only after a terminal commit, so a worker that lost its lease cannot overwrite a newer result, and a crash after commit does not re-run the handler.
- A priority window: the backlog stays in PostgreSQL ordered by priority, and only a bounded number of entries are placed in each FIFO Redis stream.
- A shutdown path that hands leases back and refunds the attempt, so deployments do not consume retry budget.
- A failure model in which every scenario is tied to an automated test (Section V), and a measured evaluation with its limits stated (Section VI), including three defects the benchmarks found.

The system guarantees at-least-once execution. It does not provide exactly-once effects on external systems, and we do not claim it.

II. Background and Related Work

A. Delivery guarantees

At-most-once delivery can lose work; exactly-once effects across a queue and an external system require the receiver to deduplicate. Queue systems therefore generally offer at-least-once delivery and ask handlers to be idempotent. Celery's documentation states this directly for late acknowledgement: a task that already started will be retried after an abrupt worker failure, so it must be idempotent [1].

B. Building blocks

Transactional outbox. Instead of writing to a database and a broker separately, the service writes the message into an outbox table in the same transaction as the state change, and a separate relay publishes it [2]. This follows the general argument that coordination across separate stores should be built from idempotent messages rather than distributed transactions [3].

Leases. A lease grants exclusive rights for a bounded time and expires on its own if the holder fails [4]. A lease alone does not stop a slow holder from acting after it expired; a monotonically checked token (fencing) in the write does [5].

Redis Streams. Streams with consumer groups give each entry to one consumer and keep it in a pending list until acknowledged. XAUTOCLAIM transfers entries idle longer than a threshold to another consumer, and claiming resets the idle time so one consumer wins [6], [7].

SKIP LOCKED. PostgreSQL's SELECT …FOR UPDATE SKIP LOCKED skips rows that cannot be locked immediately. The documentation notes the view is inconsistent,

so it is unsuitable for general work, but it avoids lock contention among consumers of a queue-like table [8].

C. Existing systems

Table I summarises documented behaviour of widely used systems on the points TaskFlow is built around. It is limited to statements in the cited documentation and is not a performance comparison; we ran no benchmark of the other systems.

III. System Design

A. Architecture

Fig. 1 shows the components. The API authenticates callers (database-backed API keys or signed JWTs), rate-limits per user and validates input. Workers run four loops in one process: a dispatcher that moves outbox rows to Redis, a worker pool that executes jobs, a scheduler that promotes delayed jobs, and a retention task that deletes old data. Prometheus scrapes the API and every worker.

B. Data model

The central tables are jobs (status, attempts, version, lease id and expiry, priority), job_attempts, job_events (append-only history), job_deliveries (the outbox), workers, dead_letter_jobs, idempotency_keys, api_keys and an operational audit log. Ten SQL migrations create them and are applied under an advisory lock with recorded checksums.

C. State machine

A job moves through the states in Fig. 2. The transition table is a single constant that is checked before every status UPDATE (12 call sites), and a contract test pins the table to the transitions the repositories actually perform. SUCCEEDED, CANCELLED and DEAD_LETTERED are terminal. A FAILED state remains defined only for legacy rows.

D. Submission and the outbox

In one PostgreSQL transaction the API inserts the job, its created/queued events and a PENDING row in job_deliveries. The API never writes the job to Redis, so a crash cannot leave a saved job without a delivery intent. An optional Idempotency-Key is checked in the same transaction under a transaction-scoped advisory lock [15]: the same key and body replays the first response, and a different body returns 409.

E. Dispatch and the priority window

The dispatcher claims due outbox rows with FOR UPDATE SKIP LOCKED and a persisted claim lease that expires if the dispatcher dies, publishes them with XADD, and records the Redis message id. A Redis stream is FIFO, so ordering must be applied before publishing. Each row stores a priority_rank computed on insert by a SQL function (so there is one definition of the order), and a partial index on (queue, rank DESC, available_at, created_at) serves the claim. Per queue and per cycle the dispatcher computes budget $= W -$ inflight, where W is the window (default 200) and inflight is pending plus undelivered entries of the consumer group, and claims at most that many rows, highest priority first. A late CRITICAL job therefore waits behind at most the window rather than the whole backlog. The window is soft: concurrent dispatchers can overshoot by one batch each, ordering is strict with no aging, and nothing is preempted.

F. Leases, fencing and commit before acknowledgement

A worker reads an entry and then acquires a lease on the job in PostgreSQL, writing a lease id and an expiry and creating an attempt. The lease is renewed at one third of its duration (30 s by default) and renewals write no events. Every result commit checks the lease id and the job version in the same transaction, so a worker whose lease expired cannot overwrite a newer outcome. The worker sends XACK only after the terminal state is committed. If it crashes in between, the entry is redelivered and the next worker finds the job already terminal and only acknowledges it, without re-running the handler.

G. Recovery

Entries left pending by a dead worker are claimed by another consumer once idle longer than the lease duration (XAUTOCLAIM). Independently, the dispatcher republishes deliveries that are marked published but whose job is not terminal after 120 s, which covers loss of Redis data. Outbox claims abandoned by a dead dispatcher become claimable again after a timeout.

H. Retries, dead letters and cancellation

Failures are classified as retriable or permanent. A retriable failure schedules a successor outbox row available after an exponentially growing, capped delay, and the job returns to QUEUED; a permanent failure or exhausted attempts moves it to DEAD_LETTERED with a dead-letter record, from which an owner may create one replacement job. Cancellation of a running job sets CANCEL_REQUESTED and signals a cooperative token. If the handler finishes at the same moment, the first durable observation wins, and a failure reported while cancellation is requested makes the worker complete the cancellation instead of scheduling a retry.

I. Scheduling, shutdown and retention

Delayed jobs are created in CREATED and promoted by the scheduler using SKIP LOCKED so concurrent schedulers handle disjoint rows. On SIGTERM a worker stops taking work, waits for running handlers up to a timeout and then hands each unfinished lease back in a fenced transaction: the job returns to QUEUED, the in-flight attempt is refunded, the outbox row is re-armed and a lease_released event is written. Without the refund a rolling deployment would spend one attempt per running job. Retention runs in every worker but is serialised by a Redis lock; it deletes terminal jobs and their history in

TABLE I
Documented behaviour of related systems (from the cited documentation only).

System	Where job state lives	Recovery after worker crash	Priority
Celery with Redis [1], [9]	Broker holds the message until acknowledged; results in an optional backend	Unacknowledged message is redelivered after the visibility timeout (default 1 hour for Redis)	Not covered here
Sidekiq [10]	Redis lists; basic fetch removes the job with BRPOP	Basic fetch can lose a job if the process crashes; Pro super_fetch keeps jobs in Redis and recovers orphans after heartbeat expiry	Queue order: strict (may starve) or weighted random
BullMQ [11], [12]	Redis	Jobs that stop renewing are moved back to waiting (a stalled job); failed after maxStalledCount (default 1)	Prioritized set; documented insertion cost $O(\log n)$
Temporal [13], [14]	Event History persisted by the Temporal service in a database	Activities retry by default with exponential backoff; a completed activity is not re-executed on replay	Not covered here
TaskFlow (this work)	PostgreSQL; Redis only notifies	Fenced lease expiry plus pending-entry reclaim; outbox republish	Priority ordered in PostgreSQL before a bounded Redis window

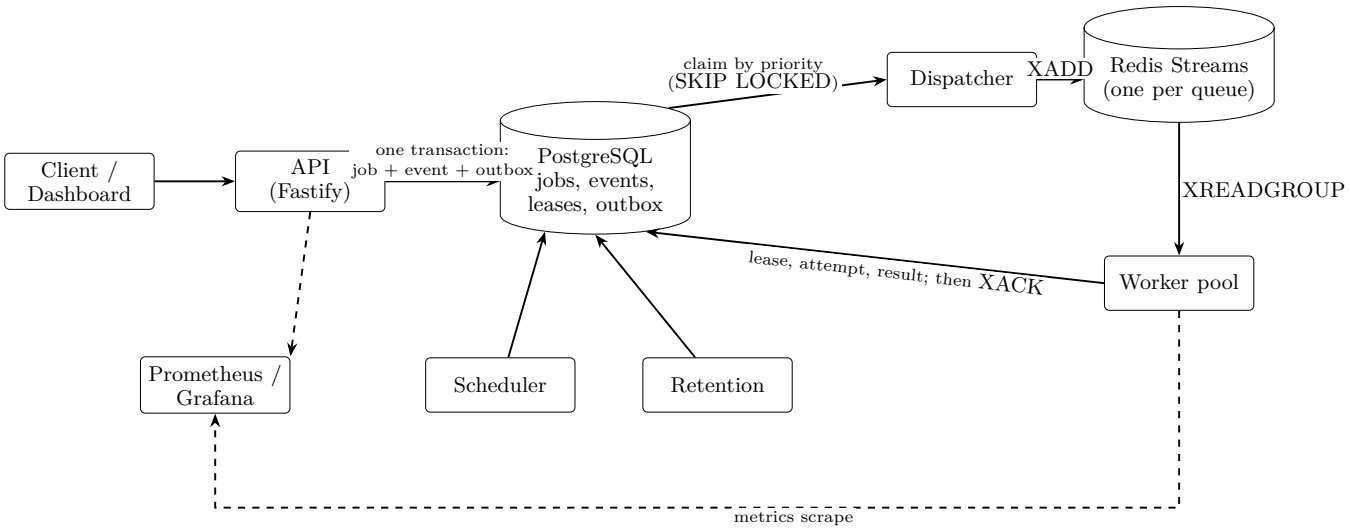


Fig. 1. Architecture. Solid arrows are the data path; dashed arrows are metrics scrapes. Redis holds no state that cannot be rebuilt from PostgreSQL.

bounded SKIP LOCKED batches, never touches data of a live job, and trims streams without passing the oldest pending entry.

IV. Implementation

The system is a pnpm and Turborepo TypeScript monorepo. The API uses Fastify; persistence uses Drizzle ORM over postgres.js with hand-written SQL migrations; Redis access uses ioredis, with a second connection reserved for blocking reads (Section VI). Metrics use prom-client, with optional OpenTelemetry tracing, and the dashboard is a React single-page application served by nginx. Measured with wc -l, the repository has 18,399 lines in 109 TypeScript/TSX source files (including 2,743 in the dashboard) and 10,271 lines in 37 test files, plus 736 lines of SQL in 10 migrations. There are 266 unit tests (plus 15 database contract tests that need PostgreSQL) and 46 Docker integration tests.

V. Failure Model and Correctness

Table II lists the failure scenarios, the intended behaviour and the integration test that exercises each. The tests start real worker processes against Dockerised PostgreSQL and Redis and inspect database rows and XPENDING directly; the names are prefixes of the test titles in tests/integration/worker-runtime.test.ts.

TABLE II
Failure scenarios and the integration tests that cover them.

Scenario	Behaviour	Test (title prefix)
No worker is running	Job stays in the outbox until a worker starts	retains an API-created job in the database outbox until a worker starts
Normal path	Terminal state committed before XACK	takes an API-created job through the shared envelope to durable SUCCEEDED before XACK
Redis entry lost	Dispatcher republishes the non-terminal job	redispatches a published nonterminal job after its Redis entry is lost
Worker SIGKILL mid-job	Entry reclaimed; handler may run twice; one idempotent effect	recovers a pending execution after SIGKILL with duplicate execution but one idempotent effect
Crash after commit, before XACK	Replacement acknowledges without re-running the handler	recognizes a result committed before crash and replacement-ACKs without rerunning the handler
Database failure during lease acquisition	Delivery stays pending and later completes	leaves a delivery pending during transient lease-database failure and later completes it
SIGTERM with running job	Job drains to SUCCEEDED before exit	drains an in-flight job to SUCCEEDED before a SIGTERM-ed worker exits
SIGTERM, handler ignores cancellation	Lease handed back, attempt budget not consumed	releases a non-cooperative job on bounded SIGTERM shutdown without consuming its attempt budget
Retries and exhaustion	Exponential delays, dead-letter only after exhaustion	derives exponential retries from execution and dead-letters only after exhaustion
Retry due while no worker runs	Retry retained through shutdown and idle interval	retains a due retry through worker shutdown and a workerless interval
Cancellation	Persisted before XACK; completed by a replacement if the owner is killed	persists cooperative cancellation before acknowledging the Redis entry; lets a replacement terminalize a cancellation request after the owner is killed
Delayed job	Promoted through scheduler and outbox	promotes a future API-created job through the canonical scheduler and outbox
Priority	Strict priority then age; late CRITICAL overtakes backlog; FIFO control with window off	releases a queued backlog strictly by priority, then by age; lets a late CRITICAL job overtake a backlog; is FIFO without the window
Queue paused	Durable delivery held, resumes later	holds durable delivery while a queue is paused and resumes processing
Retention	Active leases, pending deliveries and scheduled retries are never purged; one runner at a time	purges expired terminal jobs while strictly protecting active leases, pending deliveries, and scheduled retries; lets exactly one of two concurrent retention schedulers run, using the real Redis lock

A. Informal invariant argument

Three properties carry the design. (1) No due job without a delivery intent: every transition into QUEUED (submission, scheduler promotion, retry, release) inserts an outbox row in the same transaction, and the dispatcher keeps trying until a row is published; a published row whose job is not terminal is republished after a delay. (2) Acknowledge only after a terminal commit: XACK follows the commit in the worker, so an entry is never discarded for a job whose outcome was not stored; the converse crash (commit, no XACK) leads to a redelivery that finds a terminal job. (3) One effective lease owner: commits are conditional on the lease id and version, so at most one worker's result is accepted for an attempt even if two believe they hold the job. These are arguments over the code and tests, not a formal proof; the tests cover the listed scenarios, not all interleavings.

VI. Evaluation

A. Setup

All numbers come from one Apple M3 laptop (8 cores, 8 GB, macOS 15.6.1) running PostgreSQL 16.15 and Redis 7.4.11 in Docker (8 CPUs, 3.8 GB for the VM), Node.js 20.20.2, the built API and workers, and the k6 2.2.0 load generator on the same machine. One API process used a pool of at most 10 connections; workers ran 20 concurrent jobs each. Rate limits were raised so they did not hide the system. Jobs used either a noop handler (benchmark-only) or a process-data handler with about 50 ms of simulated work. Benchmarks used a separate database. The raw commands, scripts and caveats are in the repository (docs/benchmarking/RESULTS.md).

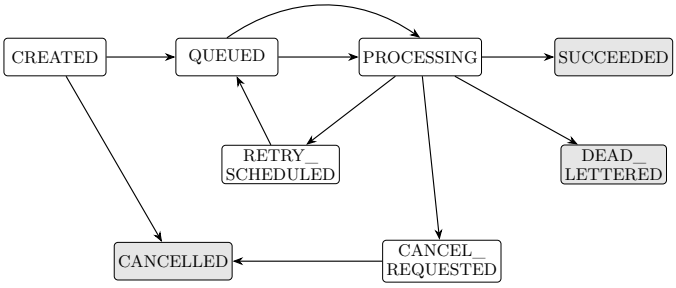


Fig. 2. Job states (main transitions). QUEUED, CREATED and RETRY_SCHEDULED can also be cancelled directly, and CANCEL_REQUESTED may still end in SUCCEEDED if the handler finishes first. The "lease handed back" arrow is PROCESSING→QUEUED.

TABLE III
API admission (60 s per rate, no workers). Errors were 0 in every run.

Target req/s	Achieved req/s	p50 ms	p95 ms	p99 ms	Dropped iter.
100	100.0	7.5	19.6	74.5	0
250	249.8	3.8	27.4	135.0	10
500	497.2	3.7	200.3	345.7	165
750	728.5	243.2	1157.7	1554.1	1033
1000	683.4	1403.1	1900.2	2417.1	18310
1500	546.7	1398.3	3079.9	3943.1	55590

B. API admission

Fig. 3 and Table III show POST /api/v1/jobs at a constant arrival rate for 60 s per rate, with no workers running (this measures the API and the transactional insert, not execution). No request failed at any rate. The stop rule was p99 above 1 s or error rate above 1%; p99 first exceeded 1 s at 750 requests/s, so the last good rate is 500 requests/s (p99 346 ms). Beyond about 700 requests/s a single API process saturates and achieved throughput falls as the target rises. The p50 at 100 requests/s is higher than at 250 because each run's first seconds include JIT warm-up, which weighs more in a smaller run.

C. Throughput versus workers

We enqueued 20,000 noop jobs with no worker running, then started N workers and timed the drain (Fig. 4). Mean throughput over three runs was 221, 241 and 203 jobs/s for 1, 2 and 4 workers. The spread within a configuration (195 to 249) is as large as the difference between configurations, so throughput did not scale with workers on this machine and the ordering of 1 and 2 workers is not significant. With the 50 ms handler and 4 workers (5,000 jobs, three runs), throughput was 184, 172 and 214 jobs/s.

PostgreSQL was the limit. During a 2-worker drain the PostgreSQL container used about 260% CPU while each worker process used about 50%; with one worker, about 170% and 77%. A job costs about 5 transactions, 4 row inserts and 7 updates (database statistics over a 3,000-job run), and sampled backends were mostly in commit or waiting on the client between statements. Setting synchronous_commit=off raised a 2-worker run only from about 270 to 297 jobs/s, so commit flushing alone does not explain the plateau. Because the backlog exists before workers start, the created-to-completed latency in these runs (p99 81 to 103 s) measures drain time, not per-job processing time.

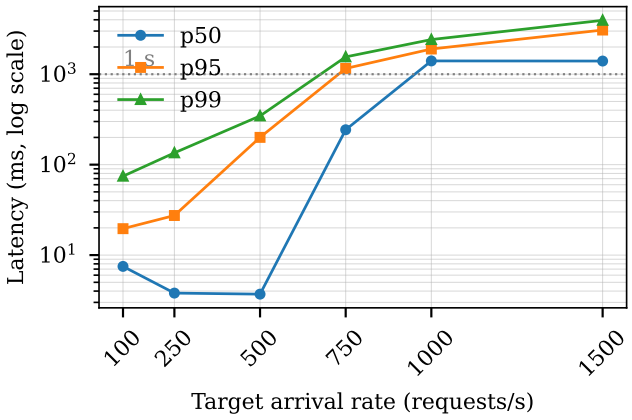


Fig. 3. Admission latency percentiles versus target arrival rate (log scale).

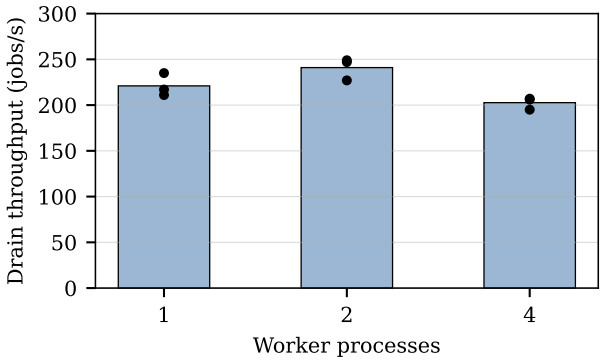


Fig. 4. Drain throughput of a 20,000-job backlog. Bars are means; dots are the three runs.

D. Crash and deployment behaviour

With 10,000 process-data jobs and two workers, one worker was killed with SIGKILL after about half the jobs had succeeded. All 10,000 jobs finished; none was lost or dead-lettered. Eight jobs were executed twice, each with a lease_expired event, and the last re-run finished 30.2 s after the kill, matching the 30 s lease. In the same setup, SIGTERM of one worker also lost no jobs, no job used more than one attempt, none was dead-lettered, and two jobs were handed back through lease_released. Each of these two experiments was run once; the earlier runs on a previous build took longer (about 60 s to finish the remainder) because of a defect described next.

E. Defects found by the benchmarks

The benchmarks exposed three defects, all fixed before the numbers above were taken.

1) Shared connection. The consumer's blocking XREADGROUP …BLOCK shared one Redis connection with the dispatcher's XADD. Each publish waited for the block to time out, so an idle worker published about 3 jobs/s. Blocking reads now use a dedicated connection. The same 1,500-job backlog went from 31 jobs in 10 s to fully drained in under 10 s.
2) Fixed sleep at full capacity. When all concurrency slots were busy the poll loop slept a fixed 250 ms, capping each worker at $20/0.25 = 80$ jobs/s whatever the handler speed. The loop now wakes when a handler settles; for the 50 ms handler with one worker, throughput rose from about 80 to 224 jobs/s.
3) Backlog-sized scan. Finding which queues had work scanned all pending rows each cycle. It now probes each active queue with EXISTS. Measured throughput did not change, so this is a cost fix, not a measured speed-up.

Regression tests cover the first two.

VII. Limitations and Future Work

- External effects. Delivery is at-least-once. A handler interrupted by a crash runs again, so side effects need their own idempotency keys. Exactly-once sinks are out of scope.
- Scale evidence. All measurements are from one machine with shared resources and default PostgreSQL settings. They show where this implementation saturates there, not production capacity, and multi-host scaling is unmeasured. The crash and deployment experiments are single runs.
- Throughput. About 5 transactions per job bound throughput on that setup. Batching outbox claims and completion writes, or tuning PostgreSQL, are untested options.
- Priority. Ordering is strict without aging, so sustained high-priority load can starve low-priority jobs; there is no preemption and priority is per queue.
- Operations. A crashed worker stays ACTIVE in the database (the dashboard counts only fresh heartbeats). The worker metrics endpoint is unauthenticated and must stay on an internal network. There is one region, no replication or backup procedure, no TLS termination, and the AWS deployment workflow and Terraform are unfinished and unused.
- Multi-tenancy. Rate limits are per user, but there is no fairness scheduling between tenants.

VIII. Conclusion

Keeping PostgreSQL as the single source of truth, writing delivery intent in the same transaction as the job, fencing every result commit with a lease, and acknowledging only after commit gave a system that lost no job in the tested crash scenarios and did not spend retry attempts on graceful shutdown. Ordering a priority backlog in PostgreSQL and exposing only a bounded window to a FIFO stream gave priority without changing the transport. The measurements also showed the cost of this durability on one machine: about 5 transactions per job and a PostgreSQL-bound plateau of roughly 200 to 250 jobs/s, and they found two defects in the worker's use of Redis that functional tests had not exposed.

References

[1] Celery Project, "Celery documentation: Optimizing (reserve one task at a time)," https://docs.celeryq.dev/en/stable/userguide/optimizing.html, accessed: Oct. 21, 2026.

[2] C. Richardson, Microservices Patterns. Manning Publications, 2018.

[3] P. Helland, "Life beyond distributed transactions: an apostate's opinion," in Proc. 3rd Biennial Conference on Innovative Data Systems Research (CIDR), 2007.

[4] C. Gray and D. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in Proc. 12th ACM Symposium on Operating Systems Principles (SOSP), 1989.

[5] M. Kleppmann, Designing Data-Intensive Applications. O'Reilly Media, 2017.

[6] Redis Ltd., "Redis documentation: Streams," https://redis.io/docs/latest/develop/data-types/streams/, accessed: Oct. 21, 2026.

[7] ——, "Redis documentation: XAUTOCLAIM," https://redis.io/docs/latest/commands/xautoclaim/, accessed: Oct. 21, 2026.

[8] PostgreSQL Global Development Group, "PostgreSQL documentation: SELECT, the locking clause," https://www.postgresql.org/docs/current/sql-select.html, accessed: Oct. 21, 2026.

[9] Celery Project, "Celery documentation: Using redis (visibility timeout)," https://docs.celeryq.dev/en/stable/getting-started/backends-and-brokers/redis.html, accessed: Oct. 21, 2026.

[10] Sidekiq, "Sidekiq wiki: Reliability," https://github.com/sidekiq/sidekiq/wiki/Reliability, accessed: Oct. 21, 2026.

[11] BullMQ, "BullMQ documentation: Stalled jobs," https://docs.bullmq.io/guide/jobs/stalled, accessed: Oct. 21, 2026.

[12] ——, "BullMQ documentation: Prioritized jobs," https://docs.bullmq.io/guide/jobs/prioritized, accessed: Oct. 21, 2026.

[13] Temporal Technologies, "Temporal documentation: Event history," https://docs.temporal.io/encyclopedia/event-history, accessed: Oct. 21, 2026.

[14] ——, "Temporal documentation: Workflow activity (activity options, event history)," https://docs.temporal.io/workflow-activity, accessed: Oct. 21, 2026.

[15] PostgreSQL Global Development Group, "PostgreSQL documentation: Explicit locking (advisory locks)," https://www.postgresql.org/docs/current/explicit-locking.html, accessed: Oct. 21, 2026.